Control de tres servomotores con Arduino

Diseño y programación en Tinkercad Circuits

Explicación del circuito

El circuito utiliza una placa Arduino Uno para controlar tres servomotores, cada uno con una forma de activación distinta. La protoboard distribuye la alimentación: el pin de 5V del Arduino se conecta a la fila positiva y el pin GND a la fila negativa. Las filas superiores e inferiores están unidas con cables, de modo que todos los componentes comparten la misma alimentación y la misma tierra. Cada servomotor tiene su cable de potencia en la fila positiva, su cable de tierra en la fila negativa y su cable de señal conectado directamente a un pin digital del Arduino.

El primer servomotor, ubicado a la izquierda, recibe su señal por el pin 11 y se activa con un pulsador conectado entre el pin 2 y GND. En el código se activa la resistencia pull-up interna del Arduino, por lo que el pin lee un nivel alto con el botón suelto y un nivel bajo al presionarlo. Cuando el programa detecta la pulsación, el servo gira a 150°, se mantiene un segundo en esa posición y regresa automáticamente a 0°. Es una acción momentánea que ocurre una vez por cada pulsación.

El segundo servomotor, ubicado en el centro, recibe su señal por el pin 10 y se controla con un switch deslizante. El terminal común del switch está conectado al pin 3 y uno de sus extremos a GND, también con la resistencia pull-up interna activada. Según la posición del switch, el servo adopta una posición fija de 30° o de 150° y se mantiene así mientras el switch no cambie, por lo que funciona como un control por estado.

El tercer servomotor, ubicado a la derecha, recibe su señal por el pin 9 y se controla con un potenciómetro. Los extremos del potenciómetro van a 5V y GND, y su terminal central a la entrada analógica A0. El Arduino lee un valor entre 0 y 1023, lo convierte en un ángulo de 0° a 180° mediante la función map(), y el servo sigue el giro de la perilla en tiempo real. Este control analógico y proporcional se diferencia claramente de los otros dos.

Finalmente, el programa usa la función millis() en lugar de delay() para medir el tiempo sin detener el Arduino. Gracias a esto, los tres servomotores pueden funcionar al mismo tiempo y de forma independiente, sin que la espera de uno interrumpa el movimiento de los demás.

Resumen de conexiones

Componente	Conexión
Servo 1 (izquierda)	Señal → pin 11 · Potencia → + · Tierra → −
Servo 2 (centro)	Señal → pin 10 · Potencia → + · Tierra → −
Servo 3 (derecha)	Señal → pin 9 · Potencia → + · Tierra → −
Pulsador	Una pata → pin 2 · Otra pata → GND
Switch deslizante	Común → pin 3 · Un extremo → GND
Potenciómetro	Central → A0 · Extremos → 5V y GND

Código del programa

```
/*
  ====================================================================
   PROYECTO: Control de tres servomotores con Arduino
   Plataforma: Tinkercad Circuits / Arduino Uno R3
  ====================================================================

   DESCRIPCIÓN GENERAL
   -------------------
   Este programa controla tres servomotores de forma independiente,
   cada uno con un tipo de activación distinto:

   a) SERVO 1 (pin 11) -> Controlado por un PULSADOR (pin 2).
      Cuando se presiona el pulsador, el servo gira de 0° a 150°,
      se mantiene ahí durante 1 segundo y luego regresa solo a 0°.
      Es una acción MOMENTÁNEA: ocurre una vez por cada pulsación.

   b) SERVO 2 (pin 10) -> Controlado por un SWITCH DESLIZANTE (pin 3).
      El servo adopta una posición fija según la posición del switch:
        - Switch conectado a GND (lectura LOW)   -> 30°
        - Switch en la otra posición (HIGH)      -> 150°
      Es un control por ESTADO: el servo se queda en esa posición
      mientras el switch no cambie.

   c) SERVO 3 (pin 9)  -> Controlado por un POTENCIÓMETRO (pin A0).
      El ángulo del servo sigue de forma continua el giro del
      potenciómetro (0° a 180°). Es un control ANALÓGICO y
      PROPORCIONAL: cada posición de la perilla corresponde a un
      ángulo diferente del servo.

   CONEXIONES
   ----------
   - Servo 1 (izquierda): Señal -> pin 11
   - Servo 2 (centro):    Señal -> pin 10
   - Servo 3 (derecha):   Señal -> pin 9
   - Los 3 servos: Potencia -> fila + (5V) | Tierra -> fila - (GND)
   - Pulsador: una pata -> pin 2 | otra pata -> GND
   - Switch deslizante: terminal común -> pin 3 | un extremo -> GND
   - Potenciómetro: extremos -> 5V y GND | terminal central -> A0

   NOTA IMPORTANTE
   ---------------
   El programa usa millis() en lugar de delay(). Así el Arduino nunca
   se queda "congelado" esperando, y los tres servos pueden responder
   al mismo tiempo (por ejemplo, mover el potenciómetro mientras el
   servo 1 está esperando en 150°).
  ====================================================================
*/

// Se incluye la librería Servo, que genera automáticamente las señales
// PWM de 50 Hz que necesitan los servomotores para posicionarse.
#include <Servo.h>

// Se crean tres objetos de tipo Servo, uno por cada servomotor.
// Cada objeto permite controlar su servo con funciones como write().
Servo servo1, servo2, servo3;

// ---------------------------------------------------------------------
// DEFINICIÓN DE PINES
// Se usan constantes (const) para que los números de pin no cambien
// por error y para que el código sea más fácil de leer y modificar.
// ---------------------------------------------------------------------
const int PIN_BOTON  = 2;    // Pin digital donde se conecta el pulsador
const int PIN_SWITCH = 3;    // Pin digital donde se conecta el switch deslizante
const int PIN_POT    = A0;   // Pin analógico donde se conecta el potenciómetro

const int PIN_SERVO1 = 11;   // Servo 1 (izquierda) -> pulsador
const int PIN_SERVO2 = 10;   // Servo 2 (centro)    -> switch
const int PIN_SERVO3 = 9;    // Servo 3 (derecha)   -> potenciómetro

// ---------------------------------------------------------------------
// VARIABLES DE CONTROL DEL SERVO 1
// ---------------------------------------------------------------------
// Indica si el servo 1 está realizando su movimiento (true) o si está
// en reposo (false). Sirve para ignorar nuevas pulsaciones mientras
// el servo todavía no ha regresado a 0°.
bool s1Activo = false;

// Guarda la lectura anterior del pulsador. Comparándola con la lectura
// actual se detecta el instante exacto en que el botón se presiona
// (detección de flanco), evitando que el movimiento se repita
// mientras el botón se mantiene presionado.
```

```
// Empieza en HIGH porque, con INPUT_PULLUP, el botón suelto lee HIGH.
bool botonPrevio = HIGH;

// Guarda el momento (en milisegundos) en que comenzó el movimiento
// del servo 1. Se usa para medir cuánto tiempo lleva en 150°.
// Se declara "unsigned long" porque millis() devuelve números muy
// grandes que no caben en un int.
unsigned long s1Inicio = 0;

// ================================================================
// SETUP: se ejecuta UNA sola vez al encender o reiniciar el Arduino.
// Aquí se configuran los pines y la posición inicial de los servos.
// ================================================================
void setup() {
  // Se configura el pin del pulsador como entrada con resistencia
  // PULL-UP interna. Esto mantiene el pin en HIGH (5 V) cuando el botón
  // está suelto, y lo lleva a LOW (0 V) cuando se presiona, porque el
  // botón lo conecta a GND. Así no se necesita resistencia externa.
  pinMode(PIN_BOTON, INPUT_PULLUP);

  // El switch también usa la resistencia PULL-UP interna:
  // - Si el switch conecta el pin 3 con GND  -> se lee LOW.
  // - Si el switch lo desconecta de GND      -> la pull-up lo pone en HIGH.
  pinMode(PIN_SWITCH, INPUT_PULLUP);

  // Se asocia cada objeto Servo con el pin digital donde está conectado
  // su cable de señal. A partir de aquí la librería empieza a enviar
  // pulsos a esos pines.
  servo1.attach(PIN_SERVO1);   // Servo 1 en el pin 11
  servo2.attach(PIN_SERVO2);   // Servo 2 en el pin 10
  servo3.attach(PIN_SERVO3);   // Servo 3 en el pin 9

  // Se colocan los servos en su posición inicial para que el sistema
  // siempre arranque desde un estado conocido.
  servo1.write(0);     // Servo 1 en reposo a 0°
  servo2.write(30);    // Servo 2 en su posición inicial de 30°
  servo3.write(0);     // Servo 3 a 0° (luego seguirá al potenciómetro)

  // Se inicia el Monitor Serie para ver mensajes de prueba.
  Serial.begin(9600);
  Serial.println("Sistema listo");
}

// ================================================================
// LOOP: se repite continuamente mientras el Arduino esté encendido.
// En cada vuelta se revisan las tres entradas y se actualizan los
// tres servos. Como no hay delay(), cada vuelta tarda muy poco tiempo.
// ================================================================
void loop() {
  // Se lee el tiempo actual en milisegundos desde que arrancó el Arduino.
  // Se guarda en una variable para usar el mismo valor en todo el ciclo.
  unsigned long ahora = millis();

  // ----------------------------------------------------------------
  // SERVO 1: CONTROL CON PULSADOR
  // Objetivo: al presionar el botón, girar a 150° y regresar a 0°
  // después de 1 segundo.
  // ----------------------------------------------------------------

  // Se lee el estado actual del pulsador:
  // HIGH = suelto, LOW = presionado (por el uso de INPUT_PULLUP).
  bool boton = digitalRead(PIN_BOTON);

  // Se inicia el movimiento solo si se cumplen TRES condiciones:
  // 1) boton == LOW           -> el botón está presionado ahora.
  // 2) botonPrevio == HIGH    -> en la vuelta anterior estaba suelto
  //                              (es decir, se ACABA de presionar).
  // 3) !s1Activo              -> el servo no está ya en movimiento.
  if (boton == LOW && botonPrevio == HIGH && !s1Activo) {
    s1Activo = true;        // Se marca que el servo 1 está en acción
    s1Inicio = ahora;       // Se guarda el momento en que empezó
    servo1.write(150);      // Se ordena al servo girar a 150°
    Serial.println("Servo 1: pulsador presionado -> 150 grados");
  }

  // Se actualiza la lectura previa para compararla en la próxima vuelta.
  botonPrevio = boton;

  // Si el servo 1 está activo y ya pasaron 1000 ms (1 segundo) desde que
  // empezó, se ordena el regreso a 0° y se vuelve al estado de reposo.
  // La resta (ahora - s1Inicio) da el tiempo transcurrido.
  if (s1Activo && ahora - s1Inicio >= 1000) {
    servo1.write(0);        // El servo regresa a su posición inicial
    s1Activo = false;       // Queda listo para una nueva pulsación
    Serial.println("Servo 1: regresa a 0 grados");
```

```
  }

  // ------------------------------------------------------------------
  // SERVO 2: CONTROL CON SWITCH DESLIZANTE
  // Objetivo: adoptar una posición distinta según la posición del switch.
  // ------------------------------------------------------------------

  // Se lee el switch en cada vuelta del loop:
  if (digitalRead(PIN_SWITCH) == LOW) {
    // Switch conectado a GND -> posición de 30°
    servo2.write(30);
  } else {
    // Switch en la otra posición (pin en HIGH por la pull-up) -> 150°
    servo2.write(150);
  }
  // Nota: enviar el mismo ángulo repetidamente no causa problemas; el
  // servo simplemente se mantiene en esa posición.

  // ------------------------------------------------------------------
  // SERVO 3: CONTROL CON POTENCIÓMETRO
  // Objetivo: que el servo siga de forma proporcional el giro de la
  // perilla del potenciómetro.
  // ------------------------------------------------------------------

  // 1) analogRead(PIN_POT) lee el voltaje del potenciómetro y lo
  //    convierte en un número entre 0 (0 V) y 1023 (5 V).
  // 2) map() convierte ese rango de 0-1023 al rango de ángulos 0-180.
  //    Ejemplo: lectura 512 (mitad) -> ángulo 90°.
  int angulo3 = map(analogRead(PIN_POT), 0, 1023, 0, 180);

  // Se envía el ángulo calculado al servo 3. Como esto ocurre en cada
  // vuelta del loop, el servo responde de inmediato al mover la perilla.
  servo3.write(angulo3);
}
```